Machine Learning

EMSI - Université Côte d'Azur
Richard Grin
Version 1.51 – 3/11/26

1

Plan de ce support

- Présentation du cours
- Généralités sur l'IA
- Machine learning
- Réseaux de neurones
- Modèles de langage
- Types de réseaux de neurones
- Transformeurs
- Références

2

Présentation du cours

- Ce cours
  Agents conversationnels en Java avec LangChain4j fait partie du parcours MIAGE « Intelligence Artificielle Appliquée » (IA2)
- Essentiellement **pratique** : comment tirer profit des modèles de langages (LLM, LM) dans les applications d'entreprise, en utilisant l'**API** de ces modèles
- Site Web du cours :
  http://richard.grin.free.fr/emsi/casablanca-ia/index.html

3

Examen

- Durée 2 heures, sans documents
- Vous choisissez la date, avec l'administration de l'EMSI
- Au moins 3 semaines après la fin du cours, pour avoir le temps de finir les TPs ; des questions pourront porter sur les TPs
- Délégués groupes 1 et 2 me tiennent au courant de la date choisie (même examen pour les 2 groupes)
- Bonus ajoutés à la note de l'examen pour avoir la note finale

4

Bonus

- Pour réactivité : réponse aux emails (en respectant format demandé), installation des logiciels, TPs terminés,…
- Réponses aux questions pendant le cours
- Questions intéressantes
- Retenez votre numéro dans la liste pour le rappeler
  - si vous avez un bonus
  - dans vos emails
- Malus pour retards, manque de travail manifeste, triche, pas pour mauvaises réponses ou questions

5

Sujet email

- Doit respecter ce format (GG = `G1` ou `G2`, nn = votre numéro dans liste) :
  [`CASA`-IA-GG] <*sujet de l'email*> - nn
  sans espaces à l'intérieur des crochets
- Par exemple
  `[CASA-IA-G1] TP 0 terminé - 12`
  email qui informe que vous avez terminé le TP `0` et que vous avez le numéro `12` dans la liste des étudiants
  ou
  `[CASA-IA-G2] Problème avec ... - 12`
  si vous demandez de l'aide

6

TPs

- **Indispensables** pour assimiler le cours
- Terminer les TPs après mon départ et avant l'examen
- Bonus pour les TPs terminés avant date limite
- Travail personnel
- Vous pouvez demander des explications à d'autres étudiants ou à une IA, échanger des idées
- **INTERDIT d'échanger du code** ou de consulter les dépôts GitHub des autres étudiants

Vous pouvez commencer le TP 1 jusqu'à
Code de l'application > Backing bean (s'arrêter juste avant « Classe JsonAdapter »)

7

Rendre un TP

- Envoyer un email, avec sujet au bon format : [CASA-IA-G2] TP 2 terminé - 12
- Email doit contenir
  - URL du dépôt GitHub **public** qui contient le code
  - **Directement** dans le texte de l'email (*pas dans un fichier joint*) les réponses aux questions posées ; ces réponses ne doivent apparaitre **nulle part ailleurs** (pas dans le README GitHub par exemple)

8

Indispensable pour bonus TPs

- Faire tous les **commits** (ou presque…)
- Répondre aux exercices ou questions posées
- Le code fourni dans les TPs ne doit pas être modifié, sauf si le TP vous le demande
  - Si vous avez une raison importante de modifier le code, demandez l'autorisation à l'enseignant, en expliquant la raison

9

Erreurs les plus communes pour les TPs

- Ne pas vérifier que les **pushs** ont été effectués
- pom.xml :
  - Mauvaise version de Java, Jakarta EE, LangChain4j
  - Plugins Maven pas à jour (aidez-vous de https://central.sonatype.com/ ou https://mvnrepository.com/)
- Pas de .gitignore pour Git
- Mauvais packages pour les classes (en particulier, **votre nom doit apparaitre dans les noms des packages**)

10

Demander de l'aide à l'enseignant ou sur le Web

- Lisez attentivement cette page : http://richard.grin.free.fr/emsi/casablanca-ia/tp/demandeaide.html
- En résumé :
  - Le minimum d'information à fournir :
    - environnement d'exécution (version de l'OS et des logiciels,…)
    - étapes qui ont conduit au problème
    - message d'erreur (et logs)
  - N'envoyez pas de copie d'écran, sauf exception

11

Utilisation de l'IA pour les TPs

- Vous pouvez utiliser l'IA **uniquement lorsque vous vous ne savez pas résoudre un problème par vous-même dans un temps raisonnable**
- Ajoutez un fichier IA.md à la racine de votre dépôt GitHub pour y indiquer comment vous avez utilisé l'IA et ce que vous en avez appris (il peut contenir « Je n'ai pas utilisé l'IA »)
- L'objectif n'est pas simplement d'obtenir une solution, mais de comprendre les explications fournies et ainsi de progresser

12

Contenu IA.md

- Pour chaque utilisation de l'IA :
  - Problème rencontré : description de la difficulté
  - Raison de l'utilisation de l'IA
  - Prompt utilisé : texte de la question posée à l'IA
  - Réponse de l'IA : réponse complète ou extrait pertinent de la conversation
  - Ce que vous avez appris : explication, avec vos propres mots, de ce que vous avez compris et retenu de cette aide

13

Abréviations dans les supports

- LM : *Language Model* ; modèle de langage ; utilisé, par exemple, par ChatGPT et Gemini
  LLM : *Large Language Model*
- BD : Base de données

14

Supports du cours

- Support 1 : Bases et concepts importants du machine learning (ML) et du deep learning (DL)
- Support 2 : Embeddings et utilisation des LMs
- Support 3 : LangChain4j, framework standard de facto pour utiliser les LMs avec du code Java ; utilisé dans TPs
- Support 4 : Fine tuning et RAG pour améliorer les réponses des LMs
- Support 5 : Introduction à l'IA agentique, protocoles MCP et A2A, skills, …

15

Parties optionnelles du cours

- Dans supports de cours et TPs
- Aucune question de l'examen ne pourra porter sur les parties optionnelles
- Utiles pour approfondissements ou aides pratiques (par exemple, pour lancer un LM localement sur son propre ordinateur)
- Dans support de cours, délimitées par ➡ (début de la partie optionnelle) et ⬆ (fin de la partie optionnelle)

16

Intelligence artificielle

17

Définitions

Exemples de tâches concrètes qui demandent de l'intelligence ?

- IA : Système informatique capable d'effectuer des tâches typiquement associées à l'intelligence, telles que l'apprentissage, le raisonnement, la résolution de problèmes, la perception ou la prise de décision (Wikipedia)
- But : automatiser, rendre plus rapides et plus efficaces des tâches qui demandent de l'intelligence
- Intelligence : capacité à comprendre, apprendre, s'adapter à des situations nouvelles, résoudre des problèmes dans un environnement donné

18

Quelques fonctions d'une IA (1/2)

- Prédiction : prévoir des résultats à partir de données ; par exemple, meilleur prix de vente d'un appartement compte tenu de ses caractéristiques
- Classification : classer des entités, classer un email comme spam, diagnostic médical
- Détection : détecter des fraudes ou une anomalie sur un objet fabriqué
- Traitement du langage : dialoguer avec des humains
- Génération : créer du contenu ; écrire un récit, générer du code informatique, des images, des vidéos

Quelques fonctions d'une IA (2/2)

- Recommandations : un site marchand recommande des produits à ses clients, compte tenu de leurs achats et comportements antérieurs sur le site
- Optimisation / planification : trouver la meilleure solution sous contraintes ; GPS
- Raisonnement : déduire, expliquer ; démontrer un théorème en suivant des règles logiques
- Interaction avec un environnement : observer un environnement, décider, agir et s'adapter ; robot, véhicule autonome

Types d'IA

- IA « classique » (ou symbolique) : un logiciel est programmé avec des règles et des connaissances fournies par des experts humains du domaine concerné ; systèmes experts
- Apprentissage automatique (ou apprentissage statistique, apprentissage machine, Machine Learning, ML) : un logiciel, appelé **modèle**, est capable d'apprendre seul à partir de données d'entrainement

Machine learning

2 phases fondamentales

- Ces 2 phases ont lieu à des moments différents

1. Durant la **phase d'apprentissage** le modèle est entrainé à accomplir des tâches, à fournir des bonnes réponses à des questions, à partir d'exemples (*dataset*)
2. Durant l'**inférence**, le modèle entrainé est utilisé pour accomplir des tâches, fournir des bonnes réponses, sur de nouvelles données

Types de machine learning

- ML « classique » : le *data scientist* choisit le type de modèle à utiliser (modèle linéaire, polynomial, arbre de décision, par exemple) pour résoudre un problème
- L'apprentissage profond (deep learning, DL) utilise le modèle des réseaux de neurones



Data scientist

- Expert en science des données
- Capable d'extraire de la valeur à partir de données brutes
- En ML, ses tâches principales sont
  - choix et préparation des données d'entrainement
  - choix du type de modèle
  - évaluation du modèle

25

Exemples types de modèles du ML classique

- Linéaire, polynomial - la valeur de sortie est calculée par une fonction linéaire, ou plus généralement polynomiale, des variables d'entrée
- Arbre de décision - utilise une structure d'arbre pour calculer la valeur de sortie ; des règles/questions déterminent le chemin dans l'arbre
- Clustering - regroupe des données en clusters, selon leur ressemblance ; par exemple, avec le « k-means clustering » on donne le nombre de groupes et le modèle définit les groupes

26

IA prédictive

- Branche du ML qui prédit un résultat à partir de données existantes, optionnellement en donnant une probabilité d'obtenir le résultat
- Exemples :
  - Prévoir si un client va acheter un produit
  - Estimer le prix d'un logement
  - Détecter une fraude à l'assurance
- Dans le ML classique les modèles sont prédictifs
- Une IA générative peut aussi être prédictive

27

IA générative (*GenAI*)

- Branche du DL qui génère des contenus nouveaux
- Les **LM**s (*Language Models*, modèles de langage) sont les logiciels d'IA générative les plus utilisés (ChatGPT, Gemini,…) ; ils génèrent des textes en langages naturels (français, anglais, …) pour des chats avec les utilisateurs, de la traduction de texte, des résumés de texte, …
- Une IA générative peut aussi générer du code, des images, de la musique, des vidéos, … ; on parlera alors d'un **LM multimodal** ou d'un **modèle de fondation**

28

Cas d'utilisation

- ML « classique », souvent avec des données structurées :
  - Prédiction de défauts de paiement (finance)
  - Classification de patients à risque (santé)
  - Décider si un email est un spam
  - Optimisation des stocks
- DL, souvent sur des données complexes, volumineuses, non structurées (textes, images, sons) :
  - Traitement du langage naturel (traduction, chatbot)
  - Vision par ordinateur (reconnaissance faciale, détection d'objets, diagnostic médical par imagerie)
  - Voitures autonomes, robots « intelligents »

29

ML classique, DL ou GenAI ?

Exemple 1 de modèle

- Modèle qui prédit si un client est susceptible d'acheter un certain produit
- Le modèle est entrainé avec de nombreux exemples **réels** de clients, avec leurs caractéristiques (âge, groupe social ou géographique, achats déjà effectués, …), en précisant si le client a acheté le produit
- Après l'entrainement, le modèle doit être capable de calculer la probabilité qu'un client achète le produit

30

Exemple 2 de modèle

- Modèle qui donne le prix d'un appartement à afficher, compte tenu de son emplacement, de sa superficie, de son étage, de son environnement, … pour qu'il trouve rapidement un acquéreur
- Pendant l'entrainement, on fournit au modèle les caractéristiques de très nombreux appartements qui ont été vendus, le prix de vente et le temps qu'il a fallu pour trouver un acquéreur

31

Exemple 3 de modèle

- Modèle qui indique si une photo contient des installations militaires
- Pendant l'entrainement, on fournit au modèle les caractéristiques de très nombreuses photos que des hommes ont classées « militaire » ou non

32

Exemple 4 de modèle

- Modèle qui génère une image à partir d'une description textuelle
- Pendant l'entrainement, on fournit au modèle des couples texte - image ; il existe des BDs (bases de données) de tels couples

33

Types d'apprentissages

- Supervisé (supervised learning, SL)
- Non supervisé (unsupervised learning, UL)
- Auto-supervisé (self-supervised learning, SSL)
- Par renforcement (reinforcement learning, RL)

34

Apprentissage supervisé

- Exemple d'apprentissage : une entrée (texte, image, caractéristiques, …), et une étiquette ou variable cible comme **valeur attendue** en sortie
- Par exemple
  - Photos de chiens et de chats étiquetées « chien » ou « chat » par des humains
  - Caractéristiques d'appartement (étage, emplacement, superficie,…) récemment vendus avec leur prix de vente

35

Apprentissage non supervisé

- Pas de valeur attendue dans les exemples d'apprentissage
- Le modèle doit trouver des patterns, des groupes ou des relations dans les données
- Pour savoir si le modèle fait du bon travail, on peut utiliser des métriques (par exemple calculer la compacité des groupes trouvés), faire appel à des d'humains experts dans le métier, comparer avec des classifications existantes



36

Exemples objectifs apprentissage non supervisés

- Extraire des classes d'individus présentant des caractéristiques communes (clustering) ; les définitions des classes ne sont pas données a priori ; par exemple, classer les clients suivant des caractéristiques qui ont un sens pour l'entreprise
- Trouver des associations entre les données ; par exemple indiquer que les clients qui achètent du pain achètent aussi souvent du beurre
- Détecter des anomalies en trouvant des items avec des caractéristiques très éloignées des autres

37

Apprentissage auto-supervisé

- Intermédiaire entre apprentissage supervisé et non supervisé
- Le modèle apprend à partir d'échantillons de données non annotées ; des « étiquettes » sont générées automatiquement à partir des données d'entrainement
- Par exemple, l'entrainement d'une IA textuelle générative peut se faire en tronquant des phrases récupérées dans un livre ou sur Internet ; l'IA doit retrouver la partie cachée qui est la variable cible

38

Apprentissage par renforcement

- Le modèle apprend en recevant un feedback (retour d'information, récompenses ou pénalités) en fonction de ses réponses ou des actions qu'il entreprend
- Le feedback peut être donné par une IA ou par un humain
- Le modèle apprend par essais et erreurs
- Souvent utilisé quand les données d'entrainement sont rares

39

Apprentissage par renforcement avec rétroaction humaine

- RLHF : Reinforcement Learning from Human Feedback
- Utilisé pour affiner les modèles, en particulier les LLMs
- Exemples d'interactions :
  - Des testeurs humains donnent leur avis sur les réponses des modèles
  - Plusieurs réponses peuvent être proposées aux testeurs par le modèle, en demandant la meilleure réponse
  - Demander aux testeurs de sélectionner avec la souris des objets dans une image



40

Apprentissage par renforcement avec récompenses vérifiables

- RLVR : Reinforcement Learning with Verifiable Rewards
- Dans certains domaines (mathématiques, logique, code avec tests unitaires, …) il est possible de vérifier automatiquement les réponses d'une IA ; la récompense utilise un vérificateur automatique plutôt que l'évaluation d'un humain

41

Etapes apprentissage LLM

1. Pré-entraînement auto-supervisé pour prédire le prochain mot ; compréhension du langage, de patterns
2. Entrainement complémentaire pour que le LLM « se comporte bien » : pas de biais, de violence, d'informations dangereuses, savoir utiliser des outils externes, …
3. Fine tuning (étudié dans autre support) avec entrainement supervisé sur des exemples pré-enregistrés de « bonnes » interactions avec un LLM
4. RLHF pour affiner encore plus les réponses du LLM

42

Distillation des connaissances

- Apparue avec les LLMs
- Technique d'apprentissage automatique qui transfère les connaissances d'un modèle « enseignant » à un modèle « élève » plus petit qui apprend à imiter les prédictions du modèle enseignant
- Modèle élève plus petit, plus rapide, moins gourmand en ressources avec connaissances proches du modèle enseignant mais avec possible perte de précision et reproduction des erreurs de l'enseignant

Modèle comme fonction

- Le modèle peut être vu comme une fonction (au sens mathématique) : $y = f(x_1, ..., x_n)$
- Il permet de trouver un résultat y à partir des données d'entrée x_i
  - Pour le type *régression*, y est numérique (valeur simple, vecteur,...)
  - Pour le type *catégoriel*, y est une catégorie (classification)
  - Pour le type *génératif*, y peut être du texte, une image, une vidéo, du son,...

Paramètres d'un modèle

- La plupart des types de modèle ont des paramètres définis à l'avance p1, p2, ... qui déterminent le comportement du modèle
- Par exemple, si on choisit un modèle linéaire, le modèle est représenté par une fonction affine qui a 2 paramètres **a** et **b** : $f(x) = \mathbf{a}x + \mathbf{b}$
- Les valeurs de ces paramètres sont déterminées durant l'apprentissage pour minimiser les erreurs de prédiction sur les données d'apprentissage

Paramètres et hyperparamètres

- Les **paramètres** d'un modèle sont *internes* au modèle ; leurs valeurs sont *automatiquement* ajustées pendant l'apprentissage ; exemple : valeurs a et b d'un modèle linéaire
- Ne pas confondre avec les **hyperparamètres** qui sont des valeurs *choisies* pour paramétrer le modèle
  - pendant l'inférence : température, taille maximale de la réponse,...
  - pendant l'apprentissage du modèle : taille des pas pendant la descente de gradient, nombre des couches cachées des réseaux de neurones, ...

Eléments pour apprentissage

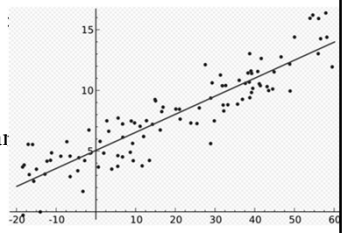
- Nombreux types d'apprentissage mais ils utilisent tous plus ou moins les mêmes éléments :
  - **Exemples d'apprentissage** ; chaque exemple est composé de valeurs qu'on donne en entrée du modèle, et du résultat attendu pour ces entrées
  - **Modèle** ; logiciel qui apprend à partir de ces exemples
  - **Fonction de perte** ; mesure l'erreur faite par le modèle lorsqu'il est testé sur un exemple d'apprentissage
  - **Algorithme d'optimisation** ; modifie le modèle pour réduire la fonction de perte
- Le but de l'apprentissage est de minimiser la fonction de perte en utilisant l'algorithme d'optimisation

Exemple d'apprentissage

- Un exemple utilisé pendant l'apprentissage est composé
  - de valeurs $x_1, ..., x_n$ en entrée (caractéristiques)
  - d'une réponse y (cible) attendue pour ces valeurs en entrée
- Pendant l'apprentissage, pour chaque exemple, le modèle est modifié pour que la **valeur calculée par le modèle** pour les x_i se rapproche de la **réponse attendue** y
- Pour éviter le bruit (un exemple peut être aberrant), on regroupe souvent les exemples dans des lots (*batchs*) de quelques dizaines à quelques centaines d'exemples, avant de modifier le modèle

Exemple régression linéaire (1/2)

- **Données d'apprentissage** : points qui correspondent aux données relevées
- **Modèle** : fonction affine f(x) = ax + b, représentée par une droite de régression

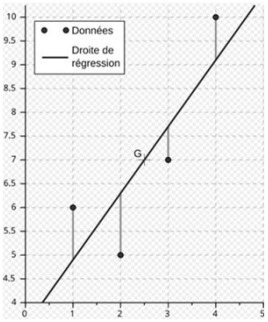


Dessin Wikipedia

A quoi correspondent les paramètres a et b pour la droite de régression ?

Exemple régression linéaire (2/2)

- **Fonction de perte** : somme des « distances » des points d'un batch d'apprentissage à la droite de régression (divisé par le nombre de points du batch)
- **Algorithme d'optimisation** : modifie les valeurs de a et de b pour minimiser la somme, par exemple avec une descente de gradient



Apprentissage

- L'apprentissage est souvent lourd et coûteux mais ne se fait qu'une seule fois
- Peut nécessiter des machines puissantes, avec GPU ou NPU (on peut en louer sur le cloud si utilisation peu fréquente)

Préparer données apprentissage

- Très important
- Supprimer données erronées et bruit (par exemple suppression balises HTML, correction fautes d'orthographe, …)
- Eliminer biais ; par exemple sur catégories de personnes (sexe, groupe ethnique, …)
- Gestion données sensibles (enlever les mots de passe, les clés secrètes, anonymiser, …)
- Ajout éventuel de contexte, par exemple pour acronymes ou références externes



Normalisation

- Cette étape permet de modifier la grandeur des valeurs pour éviter qu'elles ne soient trop grandes ou trop petites
- Elle permet, par exemple, d'éviter que les gradients deviennent trop grands ou trop petits dans la descente de gradient
- On retrouve aussi cette étape dans les transformeurs (étudiés dans la suite de ce support)



Groupes exemples apprentissage

- Exemples d'apprentissage souvent divisés en 3 groupes :
  - Groupe principal pour ajuster les paramètres du modèle ; environ 80 % des exemples
  - Groupe de validation pour vérifier la capacité du modèle à généraliser à des nouvelles situations (pas de surapprentissage) ; environ 10 % des exemples
  - Groupe de test pour tester l'efficacité du modèle final ; environ 10 % des exemples

Époque

- Exemples du groupe principal souvent utilisés plusieurs fois pendant l'apprentissage
- Une époque (*epoch*) correspond à un passage complet sur l'ensemble des exemples d'apprentissage pour améliorer le modèle
- Modifications peuvent être faites entre 2 époques ; exemples sont souvent utilisés dans un ordre différent
- Le nombre d'époques peut varier ; de quelques époques à plusieurs centaines

55

Étapes apprentissage (1/2)

- But : ajuster les paramètres pour le modèle
- On commence par donner des valeurs (aléatoires, si pas d'informations spéciales) aux paramètres du modèle
- Pour chaque exemple d'apprentissage
  - La perte, différence entre la valeur calculée par le modèle et la valeur attendue de l'exemple, dépend des paramètres du modèle (car ils interviennent dans la valeur calculée par le modèle)
  - L'algorithme d'optimisation modifie les paramètres du modèle pour minimiser la perte

56

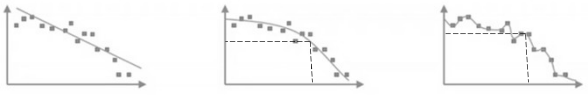
Étapes apprentissage (2/2)

- A la fin d'une époque (tous les exemples ont été utilisés), on peut s'arrêter ou commencer une nouvelle époque avec un groupe d'exemples différents, ou identiques mais pas dans le même ordre
- L'apprentissage s'arrête quand la perte sur les données de validation (souvent calculée en fin d'époque) augmente, pour éviter le surapprentissage
- Un test final sur de nouveaux exemples permet alors de valider ou non le modèle

57

Surapprentissage (overfitting)

- Il faut savoir s'arrêter à temps dans l'apprentissage
- Surapprentissage si on s'arrête trop tard : le modèle obtenu sera surentrainé sur les données d'entrainement, y compris les données aberrantes (le « bruit »), mais ne donnera pas des bons résultats avec les données nouvelles de validation



Source images : https://www.youtube.com/watch?v=kwgR-Qu56MA

58

Hyperparamètres et apprentissage

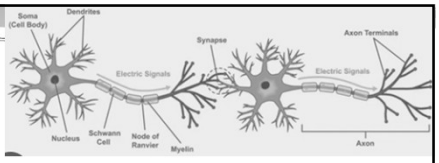
- En fait l'apprentissage est un peu plus complexe puisqu'il dépend aussi des hyperparamètres choisis pour le modèle (profondeur d'un arbre de décision, nombre de couches de neurones, taux d'apprentissage, …)
- Si le test final du modèle échoue (ou avant dans l'apprentissage), on peut être amené à changer les hyperparamètres et recommencer l'apprentissage au début

59

Réseaux de neurones

60

Cerveau humain

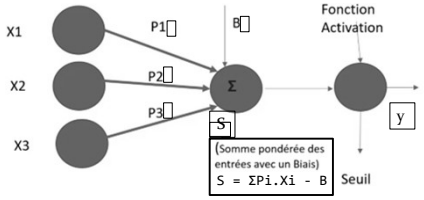


Source image : shutterstock

- 86 à 100 milliards de neurones
- Par neurone, environ 2000 dendrites (points d'entrée) ; le plus souvent un seul axone (point de sortie) mais avec plusieurs ramifications
- Chaque synapse (connexion axone-dendrite) peut être utilisée plusieurs centaines de fois par seconde
- Très efficace en énergie : consommation de 20 watts (quelques **k**watts pour une IA)
- Apprendre c'est créer, supprimer des synapses, modifier leur sensibilité

61

1ère simulation d'un neurone : le perceptron



Frank Rosenblatt, 1957

- Un perceptron a des paramètres `Pi` (**poids**) et `B` (seuil ou **biais**)
- Il reçoit n valeurs `Xi` en entrée et calcule la somme pondérée `S` de ces valeurs ; la fonction d'activation retourne la valeur finale `1` si `S - B ≥ 0`, et `0` sinon

62

Neurones artificiels

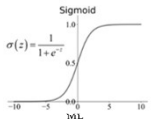
- La fonction d'activation du perceptron est trop simple et elle n'est pas continue, donc pas dérivable
- Les neurones artificiels « modernes » sont des évolutions du perceptron avec, en particulier, des fonctions d'activation dérivables qui vont permettre d'utiliser la descente de gradient

63

Fonction d'activation

- Perceptron : la sortie du perceptron est y = H(ΣPi1.Xi - B), avec H(z) = 1 si z ≥ 0, ou 0 sinon ; H a une dérivée nulle partout, sauf en 0 (pas définie)
- Neurone artificiel : on remplace H par une fonction dont la dérivée est définie presque partout
- Par exemple, avec la fonction d'activation sigmoïde (d'autres choix sont possibles), la sortie du neurone est donnée par (w_i sont les poids, b est le biais)

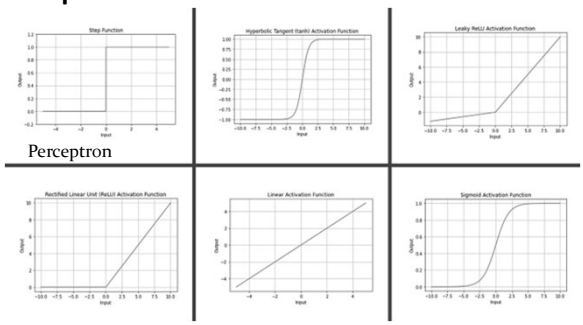
$$y = \frac{1}{1 + e^{-(\sum w_i x_i + b)}}$$



Source image : https://www.researchgate.net/publication/335845675_Reconstruction_of_porous_media_from_extremely_limited_information_using_conditional_generative_adversarial_networks

64

Graphes fonctions activation



Source image : https://cdanielaam.medium.com/activation-functions-understanding-the-different-activation-functions-in-cnns-cf5cb0c7599a

65

Types de fonctions d'activation

1. **Sigmoïde :** $\sigma(x) = 1 / (1+e^{-x})$ - Produit des sorties dans la plage de 0 à 1 ; utilisée dans les couches de sortie pour les problèmes de classification binaire
2. **Tangente hyperbolique (tanh) :** $\tanh(x) = (e^{x} - e^{-x}) / (e^{x} + e^{-x})$ - Produit des sorties dans la plage de -1 à 1, permet de gérer les valeurs négatives
3. **ReLU (Rectified Linear Unit) :** $\mathrm{ReLU}(x) = \max(0, x)$ - Renvoie simplement zéro pour les valeurs négatives et laisse les valeurs positives inchangées. Une des fonctions d'activation les plus populaires pour les couches cachées
4. **Leaky ReLU :** Leaky $\mathrm{ReLU}(x) = \max(\alpha x, x)$ - Variation de ReLU qui permet un faible taux de fuite (α est une petite valeur constante pour les entrées négatives) pour éviter certains problèmes liés à ReLU
5. **Softmax :** Souvent utilisée dans la couche de sortie pour les problèmes de classification multi-classes - convertit un vecteur de nombres réels en une distribution de probabilité

66

Réseaux de neurones artificiels

- Les neurones artificiels sont regroupés en réseaux
- Ces réseaux sont structurés en couches de neurones
- Les valeurs calculées par les neurones d'une couche sont envoyées en entrée des neurones de la couche suivante

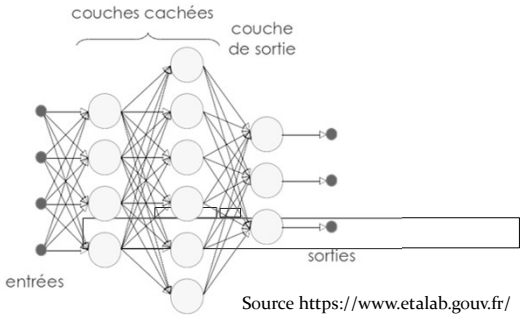
67

Couches réseau de neurones

- La couche d'entrée reçoit des valeurs de l'extérieur du réseau et les transmet aux neurones de la couche suivante
- La couche de sortie fournit à l'extérieur un résultat calculé par le réseau de neurones
- Entre ces 2 couches, les couches intermédiaires, dites cachées, participent au résultat calculé par le réseau
- Souvent les neurones d'une couche sont connectés à tous les neurones de la couche précédente et de la couche suivante

68

Exemple



Source https://www.etalab.gouv.fr/

69

Paramètres du réseau

- Poids p_{ij} de toutes les liaisons entre neurones et biais b_i des neurones
- Déterminent le comportement du réseau
- Calculés lors de la phase d'apprentissage en utilisant les exemples d'apprentissage

70

Améliorer les résultats

- Rappel : La fonction de perte évalue la qualité des résultats d'un modèle sur un exemple d'apprentissage

  Comment est calculée la fonction de perte ?

  De quoi dépend la fonction de perte ?
- L'algorithme d'apprentissage essaie de trouver les paramètres du réseau qui **minimisent** la fonction de perte
- Un des algorithmes les plus utilisés en ML est la descente de gradient

71

Gradient : généralisation de la notion de dérivée aux fonctions à plusieurs variables

Descente de gradient

- Méthode mathématique pour trouver pas à pas les valeurs des variables qui minimisent une fonction à plusieurs variables
- A chaque pas, on part des valeurs actuelles des variables de la fonction et on modifie légèrement ces valeurs pour minimiser la valeur de la fonction :
  1. Le gradient de la fonction est calculé au point qui correspond aux valeurs actuelles des variables
  2. Les nouvelles valeurs des variables sont calculées par un déplacement d'un certain pas dans la direction opposée au gradient (le gradient d'une fonction indique la direction de la plus forte augmentation de la fonction)

72

Rappels mathématiques

- Le gradient d'une fonction *f* à plusieurs variables est le vecteur de ses dérivées partielles :

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right)$$

  La valeur du gradient en un point indique la direction où la fonction augmente le plus vite autour de ce point (la « direction de la plus grande pente »)
- Soit un point (x_1, ..., x_n) ; si on veut un point proche pour lequel la valeur de la fonction sera la plus faible, on va dans la direction opposée à la direction du gradient en ce point

Descente de gradient appliquée à la fonction de perte

- La valeur de la fonction de perte pour un exemple d'apprentissage dépend des paramètres du modèle (les valeurs de l'exemple d'apprentissage sont fixes) ; les paramètres du réseau de neurones jouent le rôle des x_i
- Pour chaque batch d'apprentissage, on modifie les paramètres du modèle en appliquant la descente de gradient (on suit la plus grande pente descendante)
- A la fin de l'apprentissage les paramètres du modèle minimisent la fonction de perte (valeurs calculées par le modèle sont proches des valeurs attendues des exemples)

Rétropropagation

- Le calcul du gradient se fait par rétropropagation
- Pour chaque batch, ce calcul se fait *en remontant de la couche de sortie vers la couche d'entrée* du réseau de neurones :
  - calcul de la dérivée partielle par rapport aux paramètres de la dernière couche
  - puis on passe à la couche précédente
  - et ainsi de suite, jusqu'à arriver à la première couche

Etapes calcul des paramètres

- 2 étapes pour chaque exemple (ou groupe d'exemples) d'apprentissage :
  1. Calcul des gradients de la fonction de perte par rapport aux paramètres, par rétropropagation (*back propagation*)
  2. Ajustement des paramètres par descente de gradient, pour minimiser la fonction de perte qui, pour l'exemple d'apprentissage, calcule la distance entre
     - la valeur calculée par le réseau
     - la valeur attendue

Détails de la rétropropagation

- Fonction de perte L ; elle dépend des matrices W_i des paramètres de la couche numéro i vers la couche numéro i + 1 (on suppose n couches cachées)
  - $\partial L/\partial W_{n+1}$ est d'abord calculé (W_{n+1} contient les paramètres de la dernière couche cachée vers la couche de sortie)
  - puis $\partial L/\partial W_n$ est calculé en utilisant le résultat du calcul précédent
  - et ainsi de suite pour arriver à la fin au calcul de $\partial L/\partial W_1$ (W_1 représente les paramètres de la couche d'entrée vers la 1ère couche cachée)

Visualiser la descente de gradient

- Représenter la fonction de perte comme une hypersurface (« surface qui a une dimension de moins que l'espace auquel elle appartient ») aide à visualiser le principe de la descente de gradient

Fonction de perte représentée par une hypersurface

- La fonction de perte L dépend des n paramètres du modèle et elle fournit une seule valeur (l'erreur, la distance entre la valeur calculée et la valeur prévue)
- Elle peut donc être représentée par une hypersurface de dimension n dans un espace de dimension n + 1
- Par exemple,
  - si n = 1, une hypersurface est une courbe dans le plan ; fonction de perte de type y = L(p1)
  - si n = 2, une hypersurface est une surface dans l'espace ; fonction de perte de type y = L(p1, p2)

79

En dimension 2 (n = 1 paramètre)

- L'algorithme de la descente de gradient revient à se déplacer sur l'axe des x (donc à modifier l'unique paramètre) *en prenant la direction de la plus grande pente descendante* pour minimiser la valeur de la fonction de perte

En dimension 2 (n = 1)



Point de coordonnées (p_1, $L(p_1)$)

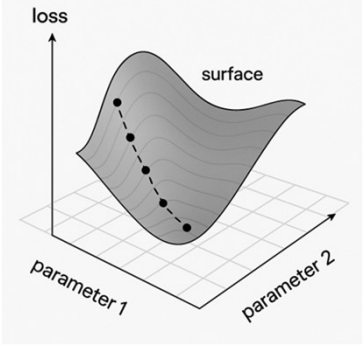
Point de coordonnées (p_2, $L(p_2)$)

Qui peut expliquer ?

Valeur **fonction de perte** en fonction valeur du paramètre

80

En dimension 3 (n = 2 paramètres)



Remarque : ne pas oublier que dans le cas d'un réseau de neurones, n est bien souvent au moins égal à **plusieurs milliards**…

Image générée par IA

81

Taux d'apprentissage

- La longueur du déplacement en direction de la plus grande pente descendante est un hyperparamètre de l'algorithme d'apprentissage, qui s'appelle le *learning rate* (taux d'apprentissage)
- Si le pas est trop petit, l'algorithme se rapproche trop lentement du minimum ; s'il est trop grand, on risque de ne pas se rapprocher du minimum (oscillations autour du minimum)
- Le pas peut être modifié en cours d'apprentissage ; il peut par exemple être diminué à chaque nouvelle époque

82

Modèles de langage

83

Définition

- Un modèle de langage (ML) est une IA entraînée sur beaucoup de textes pour pouvoir ensuite générer du texte
- On peut le considérer comme une fonction (au sens mathématique du terme) qui prend en entrée un prompt (un texte, souvent une question) et qui génère en sortie un texte qui dépend du prompt
- Utilise des réseaux de neurones

84

Textes -> nombres

- On a vu que les réseaux de neurones font des calculs sur des données **numériques** en entrée pour calculer des données en sortie
- Comment les textes manipulés par les LMs sont-ils transformés en nombres pour être traités par les réseaux de neurones ?
- Les textes sont transformés en nombres en 2 étapes :
  1. Ils sont découpés en **tokens**
  2. Chaque token est ensuite transformé en un vecteur de nombres réels appelé **embedding**

85

Token

- Unité minimale de texte traitée par un réseau de neurones
- Un token peut être un mot, mais aussi une partie de mot ou un signe de ponctuation
- Il existe aussi des tokens spéciaux ; par exemple les tokens de début et de fin de génération
- La décomposition d'un mot en tokens dépend du *tokenizer* utilisé, logiciel qui effectue le découpage

86

Vocabulaire

- Ensemble des tokens utilisés dans une langue (ou dans plusieurs langues)



87

Exemples découpage en tokens

- Tests avec plusieurs tokenizers à l'adresse https://huggingface.co/spaces/Xenova/the-tokenizer-playground ; on voit que des tokenizers différents donnent souvent des découpages différents
- « smartphone » est décomposé en 2 tokens « smart » et « phone » par la plupart des tokenizers, mais peut aussi ne pas être découpé
- dermatologue peut être découpé de multiples façons : der mat ologue, ou bien d, erm, at, olog, ue (!) par exemple

88

Intérêts des tokens

- Pourquoi découper en tokens plutôt qu'en mots ?
- Pour réduire le nombre d'unités manipulées (bien plus de mots que de tokens)
- Possible de partager les tokens entre langues
- Certaines langues n'ont pas de mots (chinois par exemple)
- Permet de gérer les mots inconnus
- Plus adapté au code informatique (par exemple, fonction de nom trierObjetsParTaille)

89



Embedding

- Chaque token est représenté par un vecteur de nombres réels, appelé embedding
- Le passage aux embeddings va permettre
  - d'effectuer des calculs mathématiques sur les données manipulées
  - mais aussi d'ajouter du sens au texte ; des tokens ayant un sens proche sont traduits par des embeddings proches (voir support de cours suivant)

90

Théorème d'approximation universelle

- Rappel : un réseau de neurones peut être considéré comme une fonction $f(x_1, ..., x_n)$
- Théorème : un réseau de neurones peut approcher une fonction quelconque d'aussi près que l'on veut
- « Un réseau de neurones à une seule couche cachée, utilisant une fonction d'activation non linéaire appropriée (comme sigmoïde, tanh, ReLU…), peut approximer n'importe quelle fonction continue sur un compact de R^n, aussi précisément que l'on veut, à condition d'avoir suffisamment de neurones dans cette couche » ; https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_d%27approximation_universelle

91

Fonction interne d'un LM

- L'entrainement du LM va permettre de trouver les paramètres pour approcher une fonction interne qui
  - prend un texte (une séquence de tokens) en entrée
  - renvoie en sortie, *pour chaque token du vocabulaire*, la probabilité que ce token convienne pour compléter le texte (selon textes donnés lors de l'apprentissage du LM)

92

Application à la génération de texte

- La fonction interne est utilisée pour la **fonction de base d'un LLM** qui
  - prend un texte en entrée
  - renvoie en sortie un des tokens les plus probables pour compléter le texte
- Pendant l'inférence, le LM va utiliser cette fonction pour générer du texte **token par token**
- Par exemple, pour répondre à une question ou pour générer la traduction d'un texte

93

Token choisi

- Le token choisi n'est pas nécessairement le token le plus probable; par exemple, le token choisi peut aussi être tiré au hasard parmi les 5 tokens les plus probables

Chacun des 5 tokens a les mêmes chances d'être choisi ?

Pourquoi le token choisi n'est pas nécessairement le plus probable ?

94

Génération de texte

- La génération de texte est **autorégressive** :
  - Le LM trouve le token qui complète la question
  - Ensuite, ce token est ajouté à la question et le LM trouve le token qui complète la question + le 1er token
  - Et ainsi de suite, jusqu'à ce que le token qui complète soit le token de fin (ou une autre condition de fin telle que « la limite de la taille de la réponse est atteinte »)

95

Exemple génération de texte pour répondre à une question

- Question « Quel est le plus grand océan du monde ? »

1. Le texte est découpé en tokens : « Quel », « est », …
2. Le modèle génère le 1er token de la réponse pour compléter la question : « **L** »
3. Puis il génère le token « ' » qui complète « Quel est le plus grand océan du monde ? **L** »
4. Et ainsi de suite, pour finalement générer « L'océan Pacifique. »
5. Pour finir, le token généré suivant est le token spécial qui indique que la génération est terminée

96

Types de réseaux de neurones

97

Types principaux de réseaux de neurones

- De convolution (CNN ; Convolutional Neural Network)
- Récurrent (RNN ; Recurrent Neural Network)
- Transformeur (utilisés par les modèles de langage)

98

CNN

- Type de réseau de neurones spécialement conçu pour traiter les **images**

99

CNN - Filtres

- Analyse l'image par petites zones pour identifier des caractéristiques importantes (contours, texture, …) en effectuant une opération mathématique appelée convolution
- Un CNN passe l'image d'entrée à travers plusieurs filtres pour détecter des caractéristiques de plus en plus complexes : d'abord des contours et textures de base, ensuite des formes géométriques, des objets, des visages
- Ces caractéristiques sont enregistrées dans des cartes de caractéristiques

100

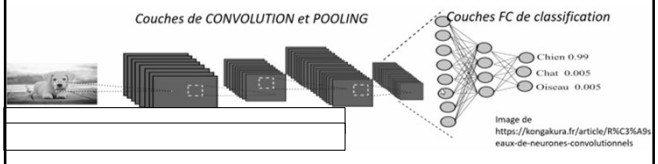
CNN - Pooling

- Permet de réduire la taille des cartes de caractéristiques en conservant les informations les plus importantes
- Consiste à diviser l'image en plusieurs petites zones (par exemple 2 x 2 pixels) et à remplacer la zone par un seul point avec une seule valeur pour chacune des caractéristiques
- La valeur du point peut être le max, la moyenne, ou être obtenue par une autre opération à partir des valeurs des pixels de la zone

101

CNN - Réseau de neurones

- Finalement, les informations extraites des filtres et du pooling sont envoyées en entrée d'un réseau de neurones pour donner une réponse ; par exemple pour savoir si une image représente un chien, un chat ou un oiseau



102

Réseaux de neurones récurrents

- Les RNNs ont été introduits pour traiter les données ordonnées en séquences : texte, image, son,..
- Le traitement utilise une chaîne de réseaux de neurones (le même réseau est utilisé dans toute la chaîne) ; à chaque étape la sortie d'un réseau de la chaîne est donnée en entrée au réseau suivant
- Exemple : 1er mot d'une phrase est donné en entrée au réseau puis on donne en entrée au 2ème réseau la sortie du 1er réseau et le 2ème mot de la phrase, etc.

103

Problèmes RNNs

- Traitement séquentiel des tokens, donc apprentissage long et moins performant pendant leur utilisation
- Difficulté à capturer les relations complexes entre les éléments de la séquence ou les relations entre éléments éloignés
- Biais temporel : les premiers éléments d'une séquence sont traités différemment des derniers
- Les transformeurs réduisent fortement ces problèmes

104

ONNX

- Open Neural Network Exchange
- Format ouvert et open source pour représenter les réseaux de neurones et de nombreux modèles de ML ; en particulier les modèles de langage
- Permet d'échanger des modèles entre différents frameworks et environnements d'exécution
- Facilite l'interopérabilité entre les outils de DL comme PyTorch, TensorFlow, etc.
- Utilisé par LangChain4j (étudié dans la suite du cours) pour intégrer des modèles d'embeddings

105

Transformeurs

106

Présentation

- Architecture pour traiter des séquences de données et utilisé actuellement par les LMs
- Peut prendre en charge plusieurs types de données : texte, image, vidéo, série temporelle,… ; dans la suite on se limitera aux textes
- Un transformeur est composé de plusieurs modules ; chacun des modules modifie les embeddings qu'il reçoit en entrée
- Certains de ces modules sont des réseaux de neurones

107

Couche d'attention

- Module important des transformeurs
- Prend en compte les relations entre les éléments de la séquence, quelle que soit leur distance dans la séquence
- Pas un réseau de neurones mais utilise aussi des paramètres définis pendant l'apprentissage
- Grâce essentiellement à cette couche d'attention, les transformeurs ont révolutionné l'IA générative
- Pourquoi « attention » : chaque embedding identifie (« porte son attention sur ») les autres embedding les plus importants pour lui, pour enrichir sa représentation

108

Motivation de l'attention

- La génération du token suivant par un LM implique une bonne compréhension du texte à compléter
- Pour cela, il ne suffit pas de comprendre chaque token isolément
- Il faut tenir compte de la syntaxe de la langue, du contexte de chaque token et des relations entre les tokens
- Les transformeurs peuvent extraire ces informations et les insérer dans les embeddings

Exemple

- « Pierre a oublié ses lunettes sur le banc sur lequel il s'était assis. »
- Un transformeur, plus précisément la couche d'attention, va capturer la relation entre « banc » et « assis », bien qu'ils soient séparés ; il a appris que ces mots sont souvent liés, parce qu'ils apparaissent souvent ensemble dans les phrases sur lesquelles on l'a entrainé
- Il va aussi associer « Pierre » et « il »

Structure d'un transformeur

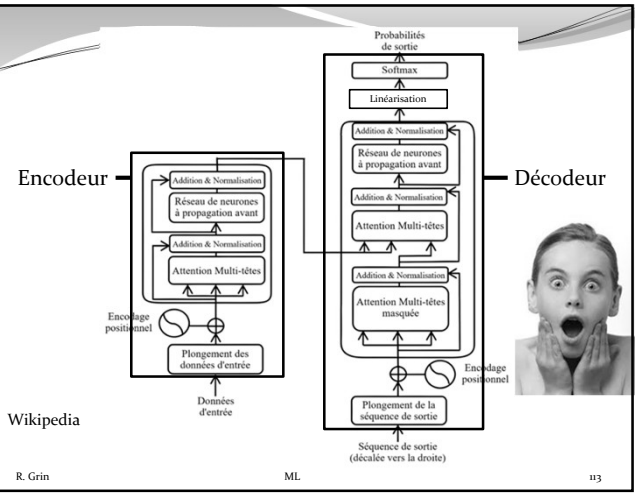
- Essentiellement 2 parties : encodeur et décodeur (en fait, une pile d'encodeurs et une pile de décodeurs)
- L'**encodeur** enrichit une séquence d'embeddings en entrée, en produisant une représentation contextuelle de chaque élément de la séquence
- Le **décodeur** génère une sortie, élément par élément

Représentation contextuelle ?

Exemple : traduction de texte

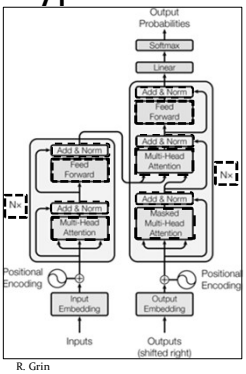
- Traduire du texte « mot à mot » (plus exactement token par token) fournit une mauvaise traduction
- Il faut 2 étapes pour une traduction acceptable :
  1. Encodeur transforme $x = (x_1, \ldots, x_n)$ en $z = (z_1, \ldots, z_n)$; les z_i sont enrichis par le contexte (relations avec les autres embeddings)
  2. Décodeur génère $y = (y_1, \ldots, y_m)$ à partir de z (le plus souvent $n \neq m$) ; processus autorégressif : y_p est généré en utilisant les y_i déjà générés ($i < p$), jusqu'à atteindre le token de fin

Exemple simple de traduction français-anglais avec $n \neq m$?



Wikipedia

Types de traitement



- Chacune de ces étapes modifie les embeddings
- Couche d'attention
- Petit réseau de neurones (Feed Forward)
- Addition et normalisation
- Remarque : « Nx » indique que le bloc est répété N fois ; N dépend du modèle (10 à plusieurs dizaines pour les LLMs)

- La suite de ce support détaille les processus qui s'exécutent dans les transformeurs ; elle est destinée à ceux qui veulent en savoir plus, mais elle ne fait pas partie du programme à connaitre pour l'examen



115

Utilité des éléments (1/2)

- **Couche d'attention** : capture dans les embeddings les **relations entre les tokens**, même s'ils sont éloignés dans la séquence
- **Réseau de neurones** (Feed-forward) : l'étape de l'attention est essentiellement linéaire ; le réseau de neurones utilise sa non-linéarité pour apprendre des informations plus complexes et enrichir **chaque embedding indépendamment** pour mieux représenter son rôle dans la phrase

116

Utilité des éléments (2/2)

- L'addition et la normalisation se font après chaque traitement
  - **Addition** : les données en entrée sont ajoutées aux données en sortie, ce qui évite qu'elles soient noyées dans les nouvelles données
  - **Normalisation** : évite des problèmes techniques durant l'apprentissage en réduisant les différences de grandeur entre les valeurs calculées ; évite en particulier « l'explosion » et « l'extinction » des gradients

117

Encodeur

- Transforme la séquence d'entrée (par exemple un texte qui représente une question, ou un texte à traduire) en une séquence d'embeddings
- Les embeddings sont traités en parallèle
- Utilise un mécanisme d'**attention** qui permet de s'intéresser plus particulièrement à certaines parties de la séquence d'entrée et de détecter les relations entre ces parties, même si elles sont éloignées dans la séquence
- Le résultat est une représentation enrichie de la séquence d'entrée (même nombre d'embeddings)

118

Décodeur

- De base, prend une séquence de tokens et sort le token suivant (itérativement une séquence de tokens : le token trouvé est ajouté à l'entrée et on cherche le token suivant)
- Si traitement préalable par encodeur, utilise la représentation enrichie de la séquence d'entrée fournie par l'encodeur pour générer une séquence de sortie
- Génère la séquence de sortie token par token, en se basant sur les informations fournies par l'encodeur, et sur les tokens qu'il a déjà générés
- Pour chaque itération, les probabilités de tous les tokens du vocabulaire sont calculées et le transformeur en choisit un (le choix dépend des hyperparamètres)

119

Encodeur ou/et décodeur

- Peuvent être utilisés indépendamment
- Modèles uniquement encodeurs : pour tâches qui nécessitent une compréhension de l'entrée, comme classification de phrases, analyse de sentiments, recherche de mots masqués
- Modèles uniquement décodeurs : pour tâches génératives telles que la génération de texte
- Modèles encodeurs-décodeurs : pour tâches génératives qui nécessitent une entrée, telles que la traduction ou le résumé de texte

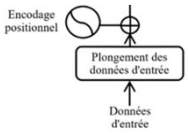
120

Exemple

- GPT-4 n'utilise qu'une pile de décodeurs
- Il n'y a pas de décodeur séparé mais, en fait, une partie du travail d'encodage est fusionnée dans les couches internes du décodeur

Travail préparatoire pour encodeur et décodeur

1. Séquence d'entrée est découpée en tokens
2. Chaque token est transformé en embedding, vecteur numérique de grande dimension
3. Tous les embeddings sont traités en parallèle ; l'encodage positionnel ajoute aux embeddings des informations sur la position des tokens



Encodage positionnel

- Un grand avantage pour l'efficacité des transformeurs par rapport aux réseaux récurrents est qu'ils peuvent traiter en parallèle tous les embeddings en entrée
- Pour ne pas perdre la position de chaque embedding dans la séquence, cette information est codée dans un vecteur de même dimension que l'embedding, et ajoutée à l'embedding

Addition et normalisation

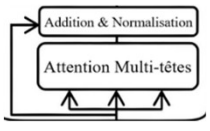
- Chaque étape importante des encodeurs et décodeurs est suivie d'une étape d'addition et de normalisation
- **Addition** : ajoute aux embeddings des informations fournies par l'étape précédente (par exemple la couche d'attention) pour que cette information ne soit pas « oubliée » car déformée par la dernière étape
- **Normalisation** : il peut y avoir des variations importantes d'amplitude dans les valeurs fournies par les différentes étapes, ce qui peut fausser le travail effectué dans les étapes suivantes ; la normalisation contrôle ces variations

Partie 1 : encodeur

Enrichit la séquence d'entrée

Multi-head (ou self-) attention

- Etape très importante qui permet à l'encodeur de capturer (« prêter attention ») à quel point chaque embedding est relié à chaque autre embedding
- A la sortie les embeddings sont enrichis avec l'attention ; par exemple un adjectif sera modifié pour tenir compte du mot qu'il qualifie



Utilité de l'étape d'attention

- Sens des mots :
  - « Je n'ai pas pu aller à mon cours de batterie car la batterie de ma voiture est tombée en panne »
  - La seule façon de distinguer les 2 sens du mot « batterie » est d'analyser le contexte qui l'entoure ; l'attention le fait (relation de « batterie » avec « cours » ou bien avec « voiture »)
- Relation entre les mots :
  - « Jean a montré à Paul où il devait signer. »
  - L'attention va permettre d'associer « il » à Paul

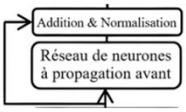
Coût de l'attention

- Chaque token est comparé à chaque autre token
- L'algorithme proposé par les auteurs de l'article « Attention is all you need » dépend donc quadratiquement de la taille du texte et, de plus, chaque comparaison nécessite des multiplications de très grandes matrices
- Ce coût important limitait fortement la taille des contextes des premiers LLMs
- Les variantes de cet algorithme utilisées maintenant réduisent cette complexité mais le traitement est encore coûteux, surtout pendant l'entrainement

Pourquoi « plusieurs têtes » ?

- On peut prêter attention à plusieurs choses dans une phrase
- De plus, une phrase peut être ambiguë : « Robert a vu Bernard avec le télescope » peut avoir 2 sens
- Une tête d'attention va associer « Robert », « a vu » et « Bernard », alors qu'une autre tête va associer « Bernard » et « avec le télescope »
- Les différentes têtes peuvent identifier différents types de relations entre les mots ; les embeddings capturent ces interprétations qui pourront être résolues par la suite avec le contexte

Réseau de neurones à propagation avant



7. Feed-forward : embeddings contextuels (enrichis) sont transmis à un petit réseau de neurones
8. Ce réseau enrichit les embeddings :
   1. Une extension linéaire augmente la dimension des embeddings en créant de nouvelles combinaisons de caractéristiques
   2. Une fonction d'activation non linéaire renforce ou atténue certaines caractéristiques
   3. Une projection finale ramène à la dimension du départ

Partie 2 : décodeur

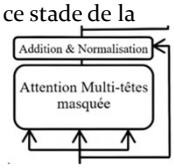
Génère la séquence de sortie, token par token, à partir de la séquence d'entrée enrichie par le travail de l'encodeur

Globalement

- Au début du traitement, la séquence de sortie du décodeur est vide (en fait, elle contient un token spécial de début de séquence) ; elle se remplit au fur et à mesure du traitement
- Celui-ci génère alors le 1er token qui est ajouté à la séquence de sortie du décodeur et on revient au début du traitement du décodeur
- Le décodeur génère ensuite le 2ème token, ajouté lui aussi à la séquence de sortie, et ainsi de suite, jusqu'à ce que le décodeur génère un token d'arrêt

Masked multi-head attention

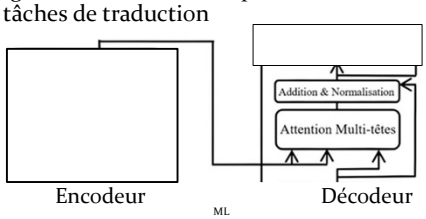
- « Self-attention »
- Comme pour l'encodeur, ajoute des informations d'attention sur la séquence de sortie déjà générée pour comprendre les relations entre ses tokens
- « Masked » car, pendant l'entrainement, on masque les tokens qui suivent le dernier token généré pour simuler ce qui va se passer pendant l'inférence : à ce stade de la génération, on ne connait pas les tokens des mots à droite du dernier mot généré
- Par souci de consistance, cette étape est conservée pendant l'inférence



133

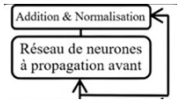
Multi-head attention

- « Cross-attention » ; différente de « self-attention » car utilise la séquence de sortie déjà générée **et** la séquence d'entrée enrichie par l'encodeur (étape éliminée si pas d'encodeur)
- Enrichit les embeddings de la sortie du décodeur en intégrant ceux de l'entrée ; particulièrement utile pour les tâches de traduction



134

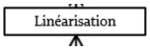
Réseau de neurones



- Feed forward (voir encodeur sur ce même thème)
- Après le dernier passage par cette couche (ne pas oublier que le bloc est répété N fois), la représentation du **dernier token** contient toutes les informations contextuelles fournies par le texte déjà généré
- Les étapes suivantes vont permettre de calculer la distribution de probabilité des éléments du vocabulaire

135

Linéarisation



- Prend en entrée l'embedding h qui représente le dernier token du texte, issu de la sortie du feed forward de la dernière répétition du bloc (annoté avec « Nx »)
- Une matrice de projection apprise transforme h en un vecteur de même dimension que le vocabulaire, qui contient les scores (appelés logits) de tous les tokens pour compléter le texte (plus le score est grand, plus le token convient pour compléter le texte)

136

Softmax, probabilités de sortie

- Softmax : Fonction qui génère des probabilités à partir des logits; indique les chances que le modèle choisisse chaque token comme prochain token dans la séquence
- Prédiction prochain token peut être faite de plusieurs manières :
  - Choisir le token le plus probable
  - Choisir le token parmi les plus probables (le choix aléatoire se fait en tenant compte des probabilités)



137

Variante « Mixture of experts »

- Variante de l'architecture classique des transformeurs, dite dense, l'architecture MoE (à mélange d'experts) remplace le module « Feed forward » par un ensemble de réseaux de neurones (appelés « experts »)
- Ces experts sont entrainés sur des problèmes particuliers
- Les experts sont déterminés automatiquement pendant l'apprentissage) ; on peut, par exemple, observer qu'un expert se spécialise dans les types de style ou de langue, ou dans les domaines scientifiques et logiques
- L'avantage de cette variante est que seulement certains experts sont activés pour chaque token ou groupe de tokens, ce qui permet de réduire le coût du calcul

138

Mode « thinking » (1/2)

- Presque toutes les dernières versions des LLMs les plus utilisés ont un mode « thinking »
- Dans ce mode, les modèles génèrent des tokens de raisonnement avant les tokens de réponse
- Ces tokens peuvent rester cachés, être résumés ou, pour certains modèles ouverts, être retournés explicitement sous forme de texte de raisonnement, selon le modèle et le paramétrage du modèle et de l'API

139

Mode « thinking » (2/2)

- Le mécanisme exact dépend des LLMs ; en particulier, les tokens de raisonnement peuvent être intégrés à la génération du token suivant par le LLM, ou bien il peut y avoir plusieurs générations de tokens (une génération préalable pour les tokens de raisonnement)
- Dans tous les cas, les tokens de raisonnement sont comptabilisés comme les tokens de réponse pour calculer le coût d'une requête

140

Références

141

IA pour les décideurs

- Présentation de l'IA pour les décideurs (2021 ; UNESCO): https://unesdoc.unesco.org/ark:/48223/pf0000380006

142

MOOCs gratuits

- L'Intelligence Artificielle… avec intelligence ! : https://www.fun-mooc.fr/fr/cours/lintelligence-artificielle-avec-intelligence/
- Intelligence artificielle pour et par les enseignants : https://www.fun-mooc.fr/fr/cours/intelligence-artificielle-pour-et-par-les-enseignants-ai4t/

143

- Lexique IA : https://www.editions-eni.fr/lexique-intelligence-artificielle-chatgpt
- Teachable machine de Google pour créer des modèles de ML rapidement et simplement, avec des images ou de l'audio, ou des poses (par exemple lever le bras) : https://teachablemachine.withgoogle.com/

144

- Lexique IA : https://www.editions-eni.fr/lexique-intelligence-artificielle-chatgpt
- Teachable machine de Google pour créer des modèles de ML rapidement et simplement, avec des images ou de l'audio, ou des poses (par exemple lever le bras) : https://teachablemachine.withgoogle.com/
- Très longue liste d'IA générative pour de multiples usages : https://codimd.apps.education.fr/s/r1H5nglAY

145

Vidéos sur IA

- Vidéo de 2 h 45 très intéressante de Yann Lecun sur l'IA en général et les limites des LLMs, en particulier des modèles non open source : https://www.youtube.com/watch?v=5t1vTLU7s40
- Dangers potentiels de l'IA ; interview de Yoshua Bengio : https://www.youtube.com/watch?v=6Zm-z032YBg
- Nombreuses vidéos et articles sur l'IA par Louis Bouchard : https://www.louisbouchard.ai/ et https://www.youtube.com/c/WhatsAI
- IA et mathématiques (Stéphane Mallat) : https://www.youtube.com/watch?v=092lytXT9-s

146

Vidéos pour machine learning

- https://www.youtube.com/channel/UCtYLUTtgS3k1Fg4y5tAhLbw ; StatQuest essaie de vulgariser les statistiques et le machine learning
- https://www.youtube.com/watch?v=trWrEWfhTVg de David Louapre, sur le deep learning
- Tour d'horizon des modèles et algorithmes en 2 vidéos, avec des descriptions rapides de chaque modèle et algorithme ; Machine Learnia, Guillaume Saint-Cirgue : https://www.youtube.com/watch?v=mT6NnslbNLM
- Apprentissage non supervisé par Machine Learnia : https://www.youtube.com/watch?v=FTtzd31IAOw

147

Vidéos pour réseaux de neurones

- https://www.youtube.com/watch?v=7ell8KEbhJ0, en français ; à voir pour commencer ; David Louapre
- https://www.youtube.com/watch?v=XUFLq6dKQok (Machine learnia), en français, une série pour étudier les algorithmes et mathématiques utilisés par IA ; le début présente le machine learning, puis le deep learning

148

- Présentation IA par AWS : https://aws.amazon.com/fr/ai/, en particulier sur overfitting : https://aws.amazon.com/fr/what-is/overfitting/
- Cours coursera sur deep learning : https://www.coursera.org/specializations/deep-learning (accès gratuit pendant 7 jours seulement)
- Hugging Face, site Web pour ceux qui travaille avec le deep learning : https://huggingface.co/

149

- https://www.youtube.com/watch?v=rn1EEd6M7fA : courte vidéo qui explique rapidement le deep learning et qui utilise TensorFlow (librairie qui permet de réaliser le réseau de neurones) et Keras (librairie pour faire du deep learning en entrainant l'IA) ; fait partie de la série « Informatique sans complexe » qui contient d'autres vidéos courtes sur l'IA et sur divers autres domaines de l'informatique : https://www.youtube.com/@InformatiqueSansComplexe

150

- Site Web Kaggle pour une communauté d'utilisateur de IA et de ML (Machine Learning). Pour apprendre (cours, guides), pour les développeurs, pour les chercheurs, et des projets sous la forme de compétitions ou autres formes. https://www.kaggle.com/

151



- Article de vulgarisation intéressant sur le phénomène de « grokking » (« au-delà du surapprentissage ») : https://scienceetonnante.substack.com/p/grokking-les-modeles-dia-sont-ils
- Claude a-t-il une conscience ? https://www.anthropic.com/research/global-workspace

152

Transformeurs (1/3)

- Cours Hugging Face : https://huggingface.co/learn/nlp-course/chapter1/1
- Visual Guide to Transformer Neural Networks : https://www.youtube.com/playlist?list=PL86uXYUJ7999zE8u2-97i4KG_2Zpufkfb
- Le transformer illustré : https://a-coles.github.io/2020/11/15/transformer-illustre.html (en français), https://jalammar.github.io/illustrated-transformer/, en vidéo : https://www.youtube.com/watch?v=-QH8fRhqFHM
- https://lbourdois.github.io/blog/nlp/Transformer/

153

Transformeurs (2/3)

- Article fondateur des transformeurs « Attention is all you need », https://arxiv.org/abs/1706.03762
- Article « The annotated transformer » qui illustre l'article avec du code en Python : https://nlp.seas.harvard.edu/2018/04/03/attention.html
- Vidéo sur transformers, par Batool Haider : https://www.youtube.com/playlist?list=PL86uXYUJ7999zE8u2-97i4KG_2Zpufkfb
- Article de Anthropic (LLM « Claude ») sur la « pensée » des LLMs : https://www.anthropic.com/news/tracing-thoughts-language-model

154

Transformeurs (3/3)

- Vidéo par un non spécialiste qui essaie de donner une intuition de ce que fait un transformeur : https://www.youtube.com/watch?v=MjCJIJ4PuSw

155